시스템 아키텍처 가이드 (Architecture Guide)

프로젝트: A-SW Hub 플랫폼 **문서번호**: ARCH-2025-ASW-001 **작성일**: 2025.09.01 **버전**: 1.0 **상태**: Approved

---

문서 개요

본 문서는 A-SW Hub 플랫폼의 고수준 시스템 아키텍처와 설계 원칙을 정의합니다. 과업 수행 시 이 가이드를 참조하여 일관된 아키텍처를 유지하시기 바랍니다.

---

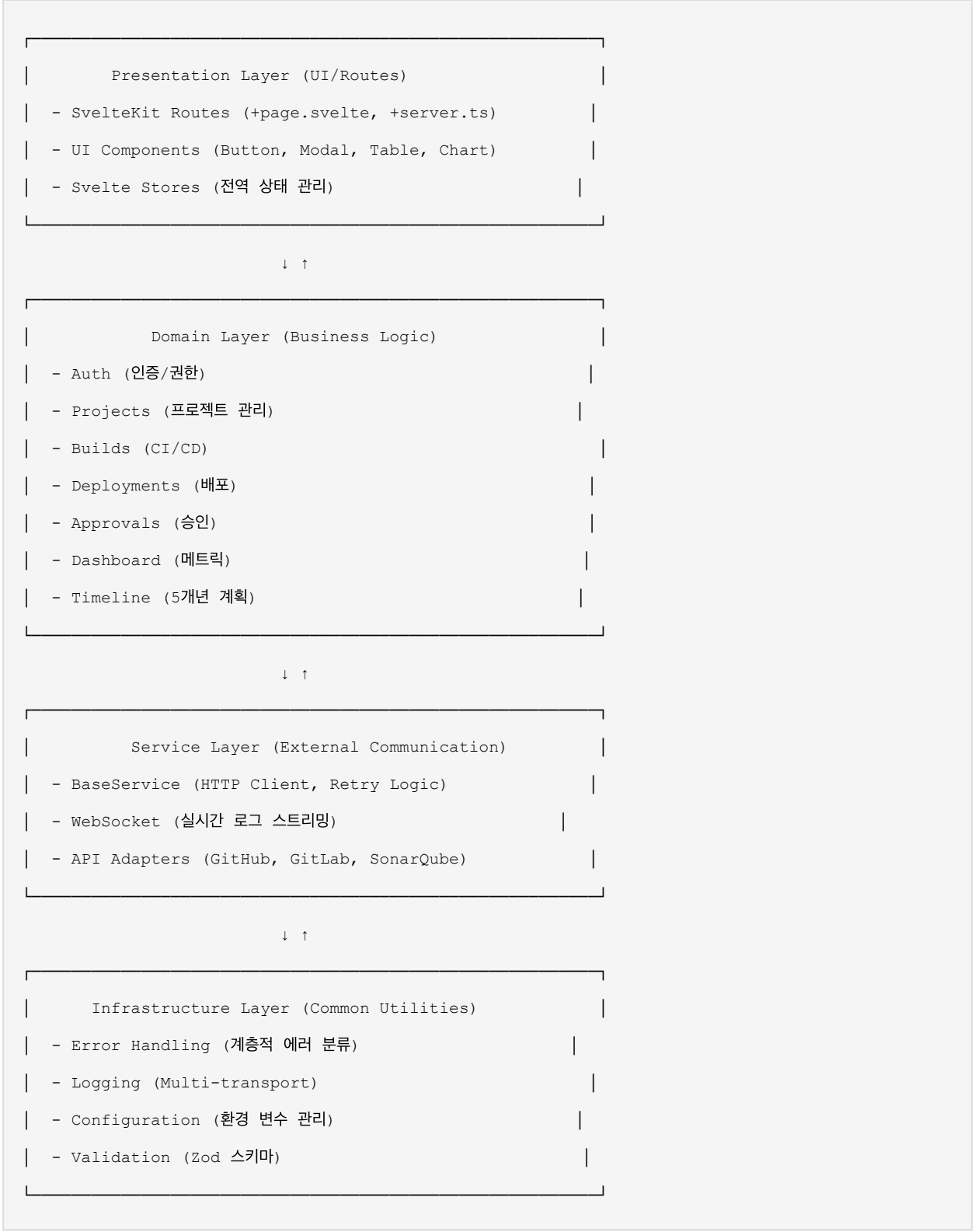
1. 아키텍처 개요

1.1 아키텍처 스타일

선택한 아키텍처: Layered Architecture + Domain-Driven Design (DDD)

1.1.1 Layered Architecture

플랫폼은 4개의 계층으로 구성되며, 각 계층은 명확한 책임과 역할을 가집니다:



1.1.2 의존성 규칙 (Dependency Rule)

계층 간 의존성은 **단방향**이어야 하며, **상위 계층은 하위 계층에만 의존**할 수 있습니다:

- ✅ **허용**: Presentation → Domain → Service → Infrastructure
- ❌ **금지**: Infrastructure → Domain, Domain → Presentation

예시:

```
// ✅ 올바른 의존성: Route (Presentation) → Service (Domain)
// src/routes/(app)/projects/+page.svelte
import { projectsStore } from '$domains/projects/store';

// ❌ 잘못된 의존성: Domain → Route
// src/domains/projects/service.ts
import SomeComponent from '$routes/(app)/projects/SomeComponent.svelte'; // 금지!
```

1.2 Domain-Driven Design (DDD) 적용

각 도메인은 **독립적인 모듈**로 설계되어, 다른 도메인과의 결합도를 최소화합니다.

1.2.1 도메인 구조

```
src/domains/{domain}/
├── types.ts          # 타입 정의 (인터페이스, Enum)
├── schemas.ts        # Zod 스키마 (런타임 검증)
├── service.ts        # 비즈니스 로직 (API 호출, 데이터 변환)
├── store.ts          # Svelte Store (상태 관리)
├── mock.ts           # Mock 데이터 (개발/테스트용)
└── utils.ts          # 도메인 특화 유틸리티
```

7개 핵심 도메인:

1. **auth**: 인증/권한 관리
2. **projects**: 프로젝트 CRUD 및 설정
3. **builds**: CI/CD 빌드 관리
4. **deployments**: 배포 관리
5. **approvals**: 승인 워크플로우
6. **dashboard**: 메트릭 집계 및 시각화
7. **timeline**: 5개년 계획 및 Gantt Chart

1.2.2 도메인 간 통신

도메인 간 직접 의존을 피하고, **이벤트** 또는 **공유 서비스**를 통해 통신합니다:

```
// ❌ 나쁜 예: builds 도메인이 projects 도메인에 직접 의존
import { getProject } from '$domains/projects/service';

// ✅ 좋은 예: Store를 통한 간접 참조
import { currentProject } from '$domains/projects/store';
const project = get(currentProject);
```

---

2. 디렉토리 구조

2.1 전체 구조

```
a-sw-hub/
├── src/
│   ├── config/                    # 중앙 집중식 설정
│   │   └── index.ts               # 환경 변수 검증 (Zod)
│   │
│   ├── lib/                       # 공유 라이브러리
│   │   ├── errors/                # 에러 계층 구조
│   │   │   ├── base.ts            # BaseError
│   │   │   ├── network.ts         # NetworkError (재시도 로직)
│   │   │   ├── validation.ts      # ValidationError
│   │   │   └── auth.ts            # AuthError (세션 갱신)
│   │   │
│   │   ├── logger/                # 다중 전송 로깅
│   │   │   ├── index.ts           # Logger 인터페이스
│   │   │   ├── console.ts         # Console Transport
│   │   │   ├── storage.ts         # LocalStorage Transport
│   │   │   └── remote.ts          # Remote Transport (프로덕션)
│   │   │
│   │   ├── services/              # 공통 서비스 레이어
│   │   │   └── base.ts            # BaseService (HTTP, Retry)
│   │   │
│   │   ├── components/            # 재사용 UI 컴포넌트
│   │   │   ├── Button.svelte
│   │   │   ├── Input.svelte
│   │   │   ├── Modal.svelte
│   │   │   ├── Table.svelte
│   │   │   ├── Chart.svelte
│   │   │   └── Gantt.svelte
│   │   │
│   │   └── types/                 # 전역 타입
│   │       ├── common.ts
│   │       └── businessPlan.ts    # 사업계획 타입
│   │
│   ├── domains/                   # 비즈니스 도메인 (DDD)
```

```
│   │   ├── auth/
│   │   ├── projects/
│   │   ├── builds/
│   │   ├── deployments/
│   │   ├── approvals/
│   │   ├── dashboard/
│   │   └── timeline/
│   │
│   ├── routes/                  # SvelteKit 라우팅
│   │   ├── (auth)/              # 인증 레이아웃
│   │   │   └── login/
│   │   │       └── +page.svelte
│   │   │
│   │   └── (app)/               # 앱 레이아웃 (인증 필요)
│   │       ├── +layout.svelte   # Sidebar, Header
│   │       ├── dashboard/
│   │       ├── projects/
│   │       │   ├── +page.svelte           # 목록
│   │       │   ├── [id]/
│   │       │   │   ├── +page.svelte       # 상세
│   │       │   │   └── settings/
│   │       │   │       └── +page.svelte   # 설정
│   │       │   └── new/
│   │       │       └── +page.svelte       # 생성
│   │       ├── builds/
│   │       ├── deployments/
│   │       ├── approvals/
│   │       ├── timeline/
│   │       └── settings/
│   │
│   ├── styles/                  # 전역 스타일
│   │   ├── global.css           # Reset, 기본 스타일
│   │   ├── variables.css        # CSS 변수
│   │   └── typography.css       # 폰트 설정
│   │
│   ├── app.html                 # HTML 템플릿
│   └── hooks.server.ts          # SvelteKit Hooks (인증 검증)
│
```

```
├── static/                      # 정적 파일
│   ├── favicon.png
│   └── robots.txt
│
├── tests/                       # 테스트
│   ├── unit/                    # Vitest 단위 테스트
│   └── e2e/                     # Playwright E2E 테스트
│
├── docs/                        # 문서
│   ├── project-management/      # 과업 문서
│   │   ├── task-order.md
│   │   ├── requirements-spec.md
│   │   ├── user-scenarios.md
│   │   └── architecture-guide.md
│   └── api/                     # API 문서
│
├── .env.example                 # 환경 변수 예시
├── svelte.config.js             # SvelteKit 설정
├── vite.config.ts               # Vite 설정
├── tsconfig.json                # TypeScript 설정 (Strict Mode)
└── package.json
```

---

3. 기술 스택

3.1 Frontend Framework

기술	버전	선정 이유
SvelteKit	2.x	- 컴파일 타임 최적화 (런타임 오버헤드 최소) - 파일 기반 라우팅 - SSR 내장 - 작은 번들 크기 (React 대비 40% 감소)
TypeScript	5.x	- 100% 타입 안정성 (Strict Mode) - 컴파일 타임 에러 차단 - IDE 자동완성 강화

기술	버전	선정 이유
Vite	5.x	- 빠른 HMR (Hot Module Replacement) - ES Modules 네이티브 지원 - 최적화된 프로덕션 빌드

3.2 State Management

기술	용도
Svelte Stores	전역 상태 관리 (Redux/Vuex 대체)
Writable Stores	읽기/쓰기 가능한 상태 (예: `currentUser`)
Derived Stores	계산된 상태 (예: `isAuthenticated`)
Readable Stores	읽기 전용 상태 (예: `appConfig`)

예시:

```
// src/domains/auth/store.ts
import { writable, derived } from 'svelte/store';

export const currentUser = writable<User | null>(null);
export const isAuthenticated = derived(currentUser, $user => $user !== null);
```

3.3 Validation & Type Safety

기술	용도
Zod	런타임 타입 검증 (API 응답, 폼 입력, 환경 변수)
TypeScript	컴파일 타임 타입 검증

타입 안정성 전략:

```
// 1. TypeScript 타입 정의
export interface Project {
  id: string;
  name: string;
  status: 'active' | 'archived';
}

// 2. Zod 스키마로 런타임 검증
import { z } from 'zod';
export const projectSchema = z.object({
  id: z.string().uuid(),
  name: z.string().min(3).max(100),
  status: z.enum(['active', 'archived'])
});

// 3. API 응답 검증
const response = await fetch('/api/projects/123');
const data = await response.json();
const project = projectSchema.parse(data); // 런타임 검증 + 타입 추론
```

3.4 Visualization

기술	용도
Chart.js	메트릭 차트 (Line, Bar, Doughnut)
Custom Gantt	타임라인 Gantt Chart (SVG 기반 자체 구현)

Black & White 테마 적용:

```
const chartOptions = {
  plugins: {
    legend: {
      labels: { color: '#000000' }
    }
  },
  scales: {
    x: { ticks: { color: '#000000' }, grid: { color: 'rgba(0,0,0,0.12)' } },
    y: { ticks: { color: '#000000' }, grid: { color: 'rgba(0,0,0,0.12)' } }
  }
};
```

3.5 Testing

기술	용도	목표 커버리지
Vitest	단위 테스트, 통합 테스트	85%
Playwright	E2E 테스트	주요 플로우 100%
MSW	API Mocking (테스트용)	-

---

4. 디자인 원칙

4.1 Extreme Black & White Industrial Design

4.1.1 색상 팔레트

단 2가지 색상만 사용:

```
:root {
  --fg: #000000;  /* Foreground (텍스트, 테두리, 아이콘) */
  --bg: #ffffff;  /* Background */

  /* 회색 표현은 투명도로 */
  --hair: rgba(0, 0, 0, 0.12);   /* Hairline 테두리 */
  --muted: rgba(0, 0, 0, 0.6);   /* 보조 텍스트 */
  --subtle: rgba(0, 0, 0, 0.04); /* 배경 강조 (Hover, Zebra) */
}
```

4.1.2 타이포그래피

```
:root {
  /* 폰트 패밀리 */
  --font-sans: 'Noto Sans KR', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  --font-mono: 'Courier New', monospace;

  /* 폰트 크기 */
  --text-xs: 11px;   /* 라벨, 캡션 */
  --text-sm: 13px;   /* 본문 보조 */
  --text-base: 14px; /* 본문 */
  --text-lg: 16px;   /* 강조 */
  --text-xl: 18px;   /* 섹션 제목 */
  --text-2xl: 24px;  /* 페이지 제목 */

  /* 폰트 두께 */
  --font-normal: 400;
  --font-medium: 500;
  --font-semibold: 600;
  --font-bold: 700;
}

/* 라벨은 항상 대문자 */
.label {
  font-size: var(--text-xs);
  font-weight: var(--font-semibold);
  text-transform: uppercase;
  letter-spacing: 0.5px;
}
```

4.1.3 UI 요소 규칙

모든 UI 요소는 다음 규칙을 따릅니다:

1. **Zero Radius**: `border-radius: 0` (모서리 둥글림 금지)
2. **No Shadows**: `box-shadow: none` (그림자 금지)
3. **Hairline Borders**: `border: 1px solid var(--hair)` (가는 테두리만)
4. **No Icons**: Lucide 아이콘 금지, ASCII 심볼 사용 (▧ ▦ ▩ ▪ ◆ ■ ▢)

예시:

```
<!-- ✅ 올바른 버튼 -->
<button class="btn-primary">
  ▪ NEW PROJECT
</button>

<style>
.btn-primary {
  background: var(--fg);
  color: var(--bg);
  border: 1px solid var(--fg);
  border-radius: 0;  /* 직각 */
  box-shadow: none;  /* 그림자 없음 */
  padding: 8px 16px;
  font-size: var(--text-sm);
  font-weight: var(--font-semibold);
  text-transform: uppercase;
  cursor: pointer;
}

.btn-primary:hover {
  background: var(--bg);
  color: var(--fg);
}
</style>
```

---

5. Configuration-Driven Architecture

5.1 환경 변수 관리

모든 설정은 **환경 변수**로 외부화하여, 코드 수정 없이 동작을 제어할 수 있습니다.

5.1.1 환경 변수 정의 (.env)

```
# API 설정
VITE_API_BASE_URL=https://api.asw-hub.com
VITE_MOCK_API=false                    # true: Mock 데이터 사용, false: 실제 API

# 인증
VITE_OAUTH_GITHUB_CLIENT_ID=Iv1.1a2b3c4d5e6f7g8h
VITE_JWT_SECRET=super-secret-key-change-in-production

# 기능 토글
VITE_ENABLE_REALTIME=true              # 실시간 로그 스트리밍
VITE_ENABLE_NOTIFICATIONS=true         # 알림 기능

# 로깅
VITE_LOG_LEVEL=debug                   # debug | info | warn | error
VITE_LOG_REMOTE=false                  # 원격 로깅 전송

# 성능
VITE_CACHE_TTL=300                     # 캐시 TTL (초)
VITE_API_TIMEOUT=30000                 # API 타임아웃 (ms)
```

5.1.2 환경 변수 검증 (src/config/index.ts)

Zod로 환경 변수를 검증하여, 런타임 에러를 사전 차단합니다:

```
import { z } from 'zod';

const configSchema = z.object({
  apiBaseUrl: z.string().url(),
  mockApi: z.boolean(),
  oauthGithubClientId: z.string().min(1),
  jwtSecret: z.string().min(32),
  enableRealtime: z.boolean(),
  enableNotifications: z.boolean(),
  logLevel: z.enum(['debug', 'info', 'warn', 'error']),
  logRemote: z.boolean(),
  cacheTTL: z.number().positive(),
  apiTimeout: z.number().positive()
});

export const config = configSchema.parse({
  apiBaseUrl: import.meta.env.VITE_API_BASE_URL,
  mockApi: import.meta.env.VITE_MOCK_API === 'true',
  oauthGithubClientId: import.meta.env.VITE_OAUTH_GITHUB_CLIENT_ID,
  jwtSecret: import.meta.env.VITE_JWT_SECRET,
  enableRealtime: import.meta.env.VITE_ENABLE_REALTIME === 'true',
  enableNotifications: import.meta.env.VITE_ENABLE_NOTIFICATIONS === 'true',
  logLevel: import.meta.env.VITE_LOG_LEVEL || 'info',
  logRemote: import.meta.env.VITE_LOG_REMOTE === 'true',
  cacheTTL: parseInt(import.meta.env.VITE_CACHE_TTL || '300', 10),
  apiTimeout: parseInt(import.meta.env.VITE_API_TIMEOUT || '30000', 10)
});

// 사용 예시
import { config } from '$config';
if (config.enableRealtime) {
  // 실시간 기능 활성화
}
```

---

6. Error Handling Strategy

6.1 계층적 에러 분류

모든 에러는 **BaseError**를 상속하며, 에러 타입에 따라 자동 처리됩니다:

```
// src/lib/errors/base.ts
export class BaseError extends Error {
  constructor(
    message: string,
    public code: string,
    public statusCode: number = 500,
    public metadata?: Record<string, unknown>
  ) {
    super(message);
    this.name = this.constructor.name;
  }
}

// src/lib/errors/network.ts
export class NetworkError extends BaseError {
  constructor(message: string, public url: string, public retryCount = 0) {
    super(message, 'NETWORK_ERROR', 503, { url, retryCount });
  }
}

// src/lib/errors/validation.ts
export class ValidationError extends BaseError {
  constructor(message: string, public fields: Record<string, string>) {
    super(message, 'VALIDATION_ERROR', 400, { fields });
  }
}

// src/lib/errors/auth.ts
export class AuthError extends BaseError {
  constructor(message: string, public reason: 'expired' | 'invalid' | 'unauthorized') {
    super(message, 'AUTH_ERROR', 401, { reason });
  }
}
```

6.2 자동 에러 처리

```
// src/lib/services/base.ts
export class BaseService {
  async request<T>(url: string, options?: RequestInit): Promise<T> {
    try {
      const response = await fetch(url, options);

      if (!response.ok) {
        if (response.status === 401) {
          throw new AuthError('Unauthorized', 'unauthorized');
        }
        throw new NetworkError(`HTTP ${response.status}`, url);
      }

      const data = await response.json();
      return data;
    } catch (error) {
      if (error instanceof AuthError) {
        // 세션 갱신 시도
        await this.refreshSession();
        return this.request<T>(url, options); // 재시도
      } else if (error instanceof NetworkError) {
        // 자동 재시도 (3회까지, Exponential Backoff)
        if (error.retryCount < 3) {
          await this.sleep(2 ** error.retryCount * 1000);
          return this.request<T>(url, options);
        }
      }
      throw error;
    }
  }
}
```

7. Logging Infrastructure

7.1 Multi-Transport Logging

개발/프로덕션 환경에 따라 로그를 다른 위치로 전송합니다:

```
// src/lib/logger/index.ts
export class Logger {
  constructor(private transports: LogTransport[]) {}

  debug(message: string, meta?: object) {
    this.log('debug', message, meta);
  }

  info(message: string, meta?: object) {
    this.log('info', message, meta);
  }

  warn(message: string, meta?: object) {
    this.log('warn', message, meta);
  }

  error(message: string, error?: Error, meta?: object) {
    this.log('error', message, { ...meta, error: error?.stack });
  }

  private log(level: LogLevel, message: string, meta?: object) {
    const entry: LogEntry = {
      timestamp: new Date().toISOString(),
      level,
      message,
      meta
    };

    this.transports.forEach(transport => transport.log(entry));
  }
}

// 환경별 설정
const logger = new Logger([
  new ConsoleTransport(),                          // 개발: 콘솔
  config.logRemote ? new RemoteTransport() : null  // 프로덕션: 원격 서버
].filter(Boolean));
```

```
export default logger;
```

---

8. 보안 설계

8.1 인증 흐름

```
[Client]                [A-SW Hub]              [GitHub OAuth]
   |                         |                         |
   |-- (1) Login Request --->|                         |
   |                         |-- (2) OAuth Redirect --->|
   |                         |                         |
   |<----- (3) Auth Code ------------------------------|
   |                         |                         |
   |-- (4) Auth Code ------->|                         |
   |                         |-- (5) Token Exchange --->|
   |                         |<-- (6) Access Token -----|
   |                         |                         |
   |<-- (7) JWT Token -------|                         |
   |    (HttpOnly Cookie)    |                         |
```

8.2 보안 체크리스트

항목	구현 방법	검증 방법
HTTPS Only	TLS 1.3 강제	HTTP → HTTPS 리다이렉트 확인
XSS 방지	Svelte 기본 Escaping	`<script>` 입력 테스트
CSRF 방지	CSRF Token	POST 요청 시 토큰 검증
SQL Injection	Prepared Statements	SQL 특수문자 입력 테스트
Rate Limiting	100 req/min per IP	초과 요청 시 429 응답
CSP	Content-Security-Policy 헤더	인라인 스크립트 차단 확인
비밀번호 해싱	bcrypt (12 rounds)	DB에 평문 저장 금지

항목	구현 방법	검증 방법
JWT Secret	환경 변수 관리	.env 파일 Git 제외

---

9. 성능 최적화 전략

9.1 번들 최적화

```
// vite.config.ts
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          // 벤더 코드 분리
          'vendor-svelte': ['svelte', 'svelte/store'],
          'vendor-chart': ['chart.js'],
          // 도메인별 청크
          'domain-projects': ['$domains/projects'],
          'domain-builds': ['$domains/builds']
        }
      }
    },
    minify: 'terser',
    terserOptions: {
      compress: {
        drop_console: true  // 프로덕션에서 console.log 제거
      }
    }
  }
});
```

9.2 코드 스플리팅

```
<script>
// 동적 임포트로 Chart.js 지연 로딩
let Chart;
onMount(async () => {
  const module = await import('chart.js');
  Chart = module.Chart;
});
</script>
```

9.3 캐싱 전략

```
// src/lib/services/base.ts
export class BaseService {
  private cache = new Map<string, CacheEntry>();

  async getCached<T>(key: string, fetcher: () => Promise<T>, ttl = 300): Promise<T> {
    const cached = this.cache.get(key);
    if (cached && Date.now() - cached.timestamp < ttl * 1000) {
      return cached.data as T;
    }

    const data = await fetcher();
    this.cache.set(key, { data, timestamp: Date.now() });
    return data;
  }
}
```

---

10. 확장성 고려사항

10.1 향후 확장 포인트

1. **백엔드 API 연동**: 현재 Mock 데이터 → 실제 API 교체
    - `src/domains/*/service.ts`의 `fetch()` 호출만 수정하면 됨

- 환경 변수 `VITE_MOCK_API=false`로 전환

2. **다국어 지원**: i18n 추가
   - `src/i18n/` 폴더 생성
   - 문자열 상수를 `$t('key')` 함수로 대체
3. **실시간 협업**: WebSocket 연동
   - `src/lib/services/websocket.ts` 활성화
   - 빌드 로그, 활동 피드에 실시간 업데이트 적용
4. **모바일 앱**: 현재 반응형 웹 → Native 앱 (React Native/Flutter)
   - API 계층이 분리되어 있어 재사용 가능

---

11. 문서 이력

버전	날짜	작성자	변경 내용
1.0	2025.09.01	VIA	최초 작성

---

12. 승인

역할	이름	서명	날짜
발주기관 담당자			2025.09.01
수행기관 담당자			2025.09.01

참고 문헌

1. SvelteKit Documentation: https://kit.svelte.dev
2. TypeScript Handbook: https://www.typescriptlang.org/docs
3. Domain-Driven Design, Eric Evans, 2003
4. Clean Architecture, Robert C. Martin, 2017